Data Formats

Video - Data Formats

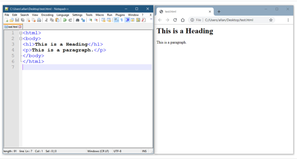
Smart devices are, in fact, tiny computers. For a smart device, such as an actuator, to react to changing conditions, it must be able to receive and interpret information sent to it by another smart device, such as a sensor. These two smart devices must share a common 'language' which is called a data format. Shared data formats are also used by other devices in the network.

Click play in the video to learn about data formats.

The Data Formats Concept

When sharing data with people, the possibilities for how to display that information are almost endless. For example, think of how a restaurant might format their menu. It could be text only, a bulleted list, or photos with captions, or just photos. These are all different ways in which the restaurant can format the data that makes up the menu. Regardless of how the menu is formatted, the information that it contains is still the same and is understandable to anyone who can read the language it is written in. This same principle applies to shared data between computers. A computer must put the data into a format that another computer can understand.

Data formats are simply a way to store and exchange data in a structured format. One such format is called Hypertext Markup Language (HTML). HTML is a standard markup language for describing the structure of web pages, as shown the figure.



These are some common data formats that are used in many applications including network automation and programmability:

- JavaScript Object Notation (JSON)
- eXtensible Markup Language (XML)
- YAML Ain't Markup Language (YAML)

The data format that is selected will depend on the format that is used by the application, tool, or script that you are using. Many systems will be able to support more than one data format, which allows the user to choose their preferred one.

Data Format Rules

Data formats have rules and structure similar to what we have with programming and written languages. Each data format will have specific characteristics:

- Syntax, which includes the types of brackets used, such as [ ], { }, ( ), the use of white space, or indentation, quotes, commas, and more.
- How objects are represented, such as characters, strings, lists, and arrays.
- How key/value pairs are represented. The key is usually on the left side and it identifies or describes the data. The value on the right is the data itself and can be a character, string, number, list or another type of data.

Search the internet for "open notify iss location now" to find a web site that tracks the current location of the International Space Station. At this web site you can see how data formats are used and some of the similarities between them. This web site includes a link for a simple Application Programming Interface (API) call to a server, which returns the current latitude and longitude of the space station along with a UNIX timestamp. The following example shows the information returned by the server using JavaScript Object Notation (JSON). The information is displayed in a raw format. This can make it difficult to understand the structure of the data.

```
{"message": "success", "timestamp": 1568789030, "iss_position": {"latitude": "25.9008", "longitude": "-132.6895"}}
```

Search the internet to find the "JSONView" browser extension or any extension that will allow you to view JSON in a more readable format. Data objects are displayed in key/value pairs. The following output shows this same output using JSONView. The key/value pairs are much easier to interpret. In the example below, you can see the key **latitude** and its value **25.9008**.

```
{
    "message": "success",
    "timestamp": 1568789030,
    "iss_position": {
        "latitude": "25.9008",
        "longitude": "-132.6895"
    }
}
```

Note: JSONView may remove the quotation marks from the key. Quotation marks are required when coding JSON key/value pairs.

Compare Data Formats

To see this same data formatted as XML or YAML, search the internet for a JSON conversion tool. At this point it is not important to understand the details of each data format, but notice how each data format makes use of syntax and how the key/value pairs are represented.

JSON Format

```
{
    "message": "success",
    "timestamp": 1568789030,
    "iss_position": {
        "latitude": "25.9008",
        "longitude": "-132.6895"
    }
}
```

YAML Format

```
message: success,
timestamp: 1568789030,
iss_position:
  latitude: "25.9008",
  longitude: "-132.6895"
```

XML Format

```
<?xml version="1.0" encoding="UTF-8" ?>
<root>
  <message>success</message>
  <timestamp>1568789030</timestamp>
  <iss_position>
    <latitude>25.9008</latitude>
    <longitude>132.6895</longitude>
  </iss_position>
</root>
```

JSON Data Format

JSON is a human readable data format used by applications for storing, transferring and reading data. JSON is a very popular format used by web services and APIs to provide public data. This is because it is easy to parse and can be used with most modern programming languages, including Python.

The following output shows an example of partial IOS output from a **show interface GigabitEthernet0/0/0** command on a router.

IOS Router Output

```
GigabitEthernet0/0/0 is up, line protocol is up (connected)
  Description: Wide Area Network
  Internet address is 172.16.0.2/24
```

This same information can be represented in JSON format. Notice that each object (each key/value pair) is a different piece of data about the interface including its name, a description, and whether the interface is enabled.

JSON Output

```
{
    "ietf-interfaces:interface": {
        "name": "GigabitEthernet0/0/0",
        "description": "Wide Area Network",
        "enabled": true,
        "ietf-ip:ipv4": {
            "address": [
                {
                    "ip": "172.16.0.2",
                    "netmask": "255.255.255.0"
                }
            ]
        }
    }
}
```

JSON Syntax Rules

These are some of the characteristics of JSON:

- It uses a hierarchical structure and contains nested values.
- It uses brackets [ ] to hold objects and square brackets [ ] hold arrays.
- Its data is written as key/value pairs.

In JSON, the data known as an object is one or more key/value pairs enclosed in braces { }. The syntax for a JSON object includes:

- Keys must be strings within double quotation marks " ".
- Values must be a valid JSON data type (string, number, array, Boolean, null, or another object).
- Keys and values are separated by a colon.
- Multiple key/value pairs within an object are separated by commas.
- Whitespace is not significant.

At times a key may contain more than one value. This is known as an array. An array in JSON is an ordered list of values. Characteristics of arrays in JSON include:

- The key followed by a colon and a list of values enclosed in square brackets [ ]
- The array is an ordered list of values.
- The array can contain multiple value types including a string, number, Boolean, object or another array inside the array.
- Each value in the array is separated by a comma.

For example, a list of IPv4 addresses might look like the following output. The key is "addresses". Each item in the list is a separate object, separated by braces { }. The objects are two key/value pairs, an IPv4 address ("ip") and a subnet mask ("netmask") separated by a comma. The array of objects in the list is also separated by a comma following the closing brace for each object.

JSON List of IPv4 Addresses

```
{
  "addresses": [
    {
      "ip": "172.16.0.2",
      "netmask": "255.255.255.0"
    },
    {
      "ip": "172.16.0.3",
      "netmask": "255.255.255.0"
    },
    {
      "ip": "172.16.0.4",
      "netmask": "255.255.255.0"
    }
  ]
}
```

YAML Data Format

YAML is another type of human readable data format used by applications for storing, transferring, and reading data. Some of the characteristics of YAML include:

- It is like JSON and is considered a superset of JSON.
- It has a minimalist format making it easy to both read and write.
- It uses indentation to define its structure, without the use of brackets or commas.

For example, look at this JSON output for a Gigabit Ethernet 2 interface.

JSON for GigabitEthernet2

```
{
    "ietf-interfaces:interface": {
        "name": "GigabitEthernet2",
        "description": "Wide Area Network",
        "enabled": true,
        "ietf-ip:ipv4": {
            "address": [
                {
                    "ip": "172.16.0.2",
                    "netmask": "255.255.255.0"
                },
                {
                    "ip": "172.16.0.3",
                    "netmask": "255.255.255.0"
                },
                {
                    "ip": "172.16.0.4",
                    "netmask": "255.255.255.0"
                }
            ]
        }
    }
}
```

That same data in YAML format is easier to read. Similar to JSON, a YAML object is one or more key value pairs. Key value pairs are separated by a colon without the use of quotation marks. In YAML, a hyphen is used to separate each element in a list. This is shown for the three IPv4 addresses in the following output.

YAML for GigabitEthernet2

```
ietf-interfaces:interface:
  name: GigabitEthernet2
  description: Wide Area Network
  enabled: true
  ietf-ip:ipv4:
    address:
    - ip: 172.16.0.2
      netmask: 255.255.255.0
    - ip: 172.16.0.3
      netmask: 255.255.255.0
    - ip: 172.16.0.4
      netmask: 255.255.255.0
```

XML Data Format

XML is one more type of human readable data format used to store, transfer, and read data by applications. Some of the characteristics of XML include:

- It is like HTML, which is the standardized markup language for creating web pages and web applications.
- It is self-descriptive. It encloses data within a related set of tags: **<tag>data</tag>**
- Unlike HTML, XML uses no predefined tags or document structure.

XML objects are one or more key/value pairs, with the beginning tag used as the name of the key: **<key>value</key>**

The following output shows the same data for GigabitEthernet2 formatted as an XML data structure. Notice how the values are enclosed within the object tags. In this example, each key/value pair is on a separate line and some lines are indented. This is not required but is done for readability. The list uses repeated instances of **<tag></tag>** for each element in the list. The elements within these repeated instances represent one or more key/value pairs.

XML for GigabitEthernet2

```
<?xml version="1.0" encoding="UTF-8" ?>
<ietf-interfaces:interface>
  <name>GigabitEthernet2</name>
  <description>Wide Area Network</description>
  <enabled>true</enabled>
  <ietf-ip:ipv4>
    <address>
      <ip>172.16.0.2</ip>
      <netmask>255.255.255.0</netmask>
    </address>
    <address>
      <ip>172.16.0.3</ip>
      <netmask>255.255.255.0</netmask>
    </address>
    <address>
      <ip>172.16.0.4</ip>
      <netmask>255.255.255.0</netmask>
    </address>
  </ietf-ip:ipv4>
</ietf-interfaces:interface>
```

Check Your Understanding - Data Formats

Check your understanding of data formats by choosing the BEST answer to the following questions.

1. Which of the following data formats is typically used to display web pages?

- (selected) HTML
- XML
- JSON
- YAML

2. Which of the following describes a key/value pair?

- A key is a string and a value is a list
- A key/value pair is another name for an array.
- (selected) A key describes the data and the value is the data itself.
- A key/value pair is only used in JSON.

3. TRUE or FALSE: White space in JSON format is significant and must be correctly formatted.

- True
- (selected) False

4. This data format is a minimalist format that is very easy to read and is considered a superset of another data format.

- HTML
- XML
- JSON
- (selected) YAML

5. This data format is self-descriptive through the use of the <tag>data</tag> structure.

- HTML
- (selected) XML
- JSON
- YAML

Check

Show Me

Reset